

NAME:K.B.Devarshan

COURSE CODE:CSA1709

REG.NO:192511426

COURSE NAME:ARTIFICIAL INTELLIGENCE

27. Write a Prolog Program to implement Best First Search algorithm

Aim:

To write a Prolog program to implement the Best First Search algorithm for finding the optimal path in a graph using heuristic evaluation.

Algorithm:

Step 1: Define the graph connections using edge facts along with their associated traversal costs.

Step 2: Define heuristic facts for each node, representing the estimated cost from that node to the goal.

Step 3: Initialize a search queue containing the starting node as the initial path.

Step 4: Define the base case where the first node in the queue is the goal node, and return the recorded path.

Step 5: For the recursive step, extract the current node from the front of the queue and find all its unvisited neighbor nodes.

Step 6: Append these neighbor nodes to the current path to form new paths, and add them to the queue.

Step 7: Sort the queue based on the heuristic values of the leading nodes of each path, ensuring the most promising node is explored next.

Step 8: Recursively call the search function with the newly sorted queue until the goal is found.

Prolog:

edge(a, b, 2).

edge(a, c, 5).

edge(b, d, 4).

edge(c, g, 1).

edge(d, g, 3).

heuristic(a, 6).

heuristic(b, 4).

heuristic(c, 1).

heuristic(d, 3).

heuristic(g, 0).

best_first(Start, Goal, Path) :-

 bfs_queue([[Start]], Goal, RevPath),

 reverse(RevPath, Path).

bfs_queue([[Goal|Rest]|_], Goal, [Goal|Rest]).

bfs_queue([[Current|Rest]|Queue], Goal, FinalPath) :-

 findall([Next, Current|Rest], (edge(Current, Next, _), \+ member(Next, [Current|Rest])),
 Children),

 append(Queue, Children, NewQueue),

 sort_queue(NewQueue, SortedQueue),

 bfs_queue(SortedQueue, Goal, FinalPath).

sort_queue([], []).

sort_queue([Head|Tail], Sorted) :-

 sort_queue(Tail, SortedTail),

 insert_sorted(Head, SortedTail, Sorted).

insert_sorted(X, [], [X]).

insert_sorted([Node1|Rest1], [[Node2|Rest2]|Tail], [[Node1|Rest1], [Node2|Rest2]|Tail]) :-

 heuristic(Node1, H1),

 heuristic(Node2, H2),

 H1 =< H2.

insert_sorted([Node1|Rest1], [[Node2|Rest2]|Tail], [[Node2|Rest2]|SortedTail]) :-

```

heuristic(Node1, H1),
heuristic(Node2, H2),
H1 > H2,
insert_sorted([Node1|Rest1], Tail, SortedTail).

```

OUTPUT:

```

GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/DELL/OneDrive/Desktop/edge.pl for byte code...
C:/Users/DELL/OneDrive/Desktop/edge.pl compiled, 30 lines read - 3895 bytes written, 46 ms
| ?- | ?- best_first(a, g, Path).
uncaught exception: error(syntax_error('user_input:1 (char:1) expression expected'),read_term/3)
| ?-
Path = [a,c,g]

yes
| ?- best_first(a, g, Path).
uncaught exception: error(syntax_error('user_input:5 (char:1) . or operator expected after expression'),read_t
| ?-
Path = [a,c,g]

yes

```

28. Write the prolog program for Medical Diagnosis

Aim:

To write a Prolog program for an expert system that diagnoses medical conditions based on patient symptoms.

Algorithm:

Step 1: Define a knowledge base of facts mapping specific diseases to their characteristic symptoms.

Step 2: Define a list of known diseases within the system.

Step 3: Create a diagnostic rule that accepts a list of symptoms provided by a patient.

Step 4: Iterate through the patient's symptom list and check if every symptom is associated with a specific disease in the knowledge base.

Step 5: If all input symptoms perfectly align with a known disease's symptoms, successfully match and return that disease as the diagnosis.

Prolog

```

symptom(flu, fever).
symptom(flu, cough).
symptom(flu, chills).

```

symptom(cold, sneezing).
symptom(cold, runny_nose).
symptom(cold, mild_cough).
symptom(measles, fever).
symptom(measles, rash).
symptom(measles, red_eyes).

disease_match(flu).
disease_match(cold).
disease_match(measles).

diagnose(Disease, SymptomList) :-
 disease_match(Disease),
 check_symptoms(Disease, SymptomList).

check_symptoms(_, []).
check_symptoms(Disease, [Symptom|Rest]) :-
 symptom(Disease, Symptom),
 check_symptoms(Disease, Rest).

OUTPUT:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

compiling C:/Users/DELL/OneDrive/Desktop/symptom(flu, fever).pl for byte code...
C:/Users/DELL/OneDrive/Desktop/symptom(flu, fever).pl compiled, 21 lines read - 2176 bytes written, 42 ms
| ?- | ?- diagnose(Disease, [fever, sneezing]).
uncaught exception: error(syntax_error('user_input:1 (char:1) expression expected'),read_term/3)
| ?- diagnose(Disease, [fever, sneezing]).
no
| ?- |
```

29. Write a Prolog Program for forward Chaining. Incorporate required queries.

Aim:

To write a Prolog program to implement forward chaining, inferring new facts from existing data iteratively until no new information can be deduced.

Algorithm:

Step 1: Declare a dynamic predicate to allow the program to assert (add) new facts into the knowledge base during runtime.

Step 2: Define the initial base facts representing known truths.

Step 3: Define rules consisting of a conclusion and a list of required premise conditions.

Step 4: Implement a forward chaining mechanism that continuously scans the rules.

Step 5: If a rule's premises are completely satisfied by existing facts, but its conclusion is not yet a fact, assert the conclusion as a new fact and print it.

Step 6: Recursively loop this process until a complete scan yields no new facts.

Prolog

```
:- dynamic fact/1.
```

```
fact(has_hair(tom)).
```

```
fact(eats_meat(tom)).
```

```
fact(has_spots(tom)).
```

```
rule(is_mammal(X), [has_hair(X)]).
```

```
rule(is_carnivore(X), [eats_meat(X)]).
```

```
rule(is_cheetah(X), [is_mammal(X), is_carnivore(X), has_spots(X)]).
```

```
check_premises([]).
```

```
check_premises([P|Rest]) :-
```

```
    fact(P),
```

```
    check_premises(Rest).
```

```
forward_chain :-
```

```

rule(Conclusion, Premises),
\+ fact(Conclusion),
check_premises(Premises),
asserta(fact(Conclusion)),
write('Derived new fact: '), write(Conclusion), nl,
forward_chain.
forward_chain :-
    write('Forward chaining complete.').

```

Required Queries:

Plaintext

```

?- forward_chain.
Derived new fact: is_mammal(tom)
Derived new fact: is_carnivore(tom)
Derived new fact: is_cheetah(tom)
Forward chaining complete.
true.

```

```

?- fact(is_cheetah(tom)).
true.

```

30. Write a Prolog Program for backward Chaining. Incorporate required queries.

Aim:

To write a Prolog program to implement backward chaining, starting from a goal and working backward to verify if supporting facts exist in the knowledge base.

Algorithm:

Step 1: Define the basic facts representing known truths in the environment.

Step 2: Define logical rules that establish conditional relationships (e.g., X is an ancestor of Y if X is the parent of Z and Z is the ancestor of Y).

Step 3: Formulate a query representing the specific goal to be proven.

Step 4: Rely on Prolog's native backward chaining inference engine to automatically locate a rule matching the goal.

Step 5: Recursively set the premises of that rule as new sub-goals until all sub-goals resolve cleanly to known base facts.

Prolog

```
parent(john, mary).
```

```
parent(mary, susan).
```

```
parent(mary, tom).
```

```
parent(susan, anna).
```

```
ancestor(X, Y) :-
```

```
    parent(X, Y).
```

```
ancestor(X, Y) :-
```

```
    parent(X, Z),
```

```
    ancestor(Z, Y).
```

Required Queries:

Plaintext

```
?- ancestor(john, anna).
```

```
true.
```

```
?- ancestor(mary, X).
```

```
X = susan ;
```

```
X = tom ;
```

```
X = anna.
```

?- ancestor(X, tom).

X = mary ;

X = john.

Output:

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diez

compiling C:/Users/DELL/OneDrive/Desktop/parent(john, mary).pl for byte code...
C:/Users/DELL/OneDrive/Desktop/parent(john, mary).pl compiled, 10 lines read - 952 bytes written, 64 ms
| ?- ancestor(john, anne).

true ?

(16 ms) yes
| ?- ancestor(mary, X).

X = susan ?

yes
| ?- ancestor(X, tom).

X = mary ?

yes
| ?- |
```

31.Create a Web Blog using Word press to demonstrate Anchor Tag, Title Tag, etc.

Aim:

To create a web blog using WordPress to demonstrate the application of foundational HTML tags such as the Anchor tag, Heading tags, and Paragraph tags.

Algorithm:

Step 1: Open a web browser, navigate to your WordPress admin login page, and enter your credentials to access the Dashboard.

Step 2: On the left-hand navigation menu, click on Posts and then select Add New to create a blank blog post.

Step 3: Type a heading into the Add Title field, which WordPress automatically formats as the main title tag (H1) for the page.

Step 4: Click the plus (+) icon in the block editor, search for Custom HTML, and add the block to the post.

Step 5: Inside the Custom HTML block, write standard HTML code incorporating heading tags, paragraph tags, and anchor tags with the href attribute.

Step 6: Use the title attribute within the anchor tag to provide tooltip text when a user hovers over the hyperlink.

Step 7: Click the Preview button to verify that the HTML tags render the links and text formats correctly.

Step 8: Click the Publish button to make the blog post live on the internet.

HTML:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>SEO Optimized Page for Anchor Tags and Titles</title>
```

```
  <meta name="description" content="Learn how to optimize anchor tags and titles  
for better search engine optimization.">
```

```
  <link rel="stylesheet" href="styles.css">
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
    <h1>SEO Best Practices: Optimizing Anchor Tags and Titles</h1>
```

```
  </header>
```

```
  <nav>
```

```
    <ul>
```

```
      <li><a href="#importance-of-titles" title="Importance of Titles in  
SEO">Importance of Titles</a></li>
```

```
      <li><a href="#optimizing-anchor-tags" title="How to Optimize Anchor  
Tags">Optimizing Anchor Tags</a></li>
```

```
      <li><a href="#best-practices" title="Best Practices for SEO">Best  
Practices</a></li>
```

```
    </ul>
```

```
  </nav>
```

```
<main>

  <section id="importance-of-titles">

    <h2>Importance of Titles in SEO</h2>

    <p>Titles are one of the most important on-page SEO element. Make sure your
titles are descriptive, contain relevant keywords, and are unique for each page.</p>

  </section>

  <section id="optimizing-anchor-tags">

    <h2>How to Optimize Anchor Tags</h2>

    <p>Anchor tags, or links, are crucial for SEO as they help search engines
understand the context of the linked page. Use descriptive and keyword-rich anchor
text for your links. Avoid generic phrases like "click here" or "read more".</p>

    <p>Example of a well-optimized anchor tag: <a href="https://example.com/seo-
tips" title="SEO Tips and Tricks">SEO Tips and Tricks</a></p>

  </section>

  <section id="best-practices">

    <h2>Best Practices for SEO</h2>

    <ul>

      <li>Use relevant keywords in your titles and anchor texts.</li>

      <li>Keep your titles under 60 characters to avoid truncation in search
results.</li>

      <li>Create unique titles for each page to avoid duplicate content issues.</li>

      <li>Use descriptive and specific anchor texts to provide context.</li>

      <li>Ensure your links are relevant and add value to the content.</li>

    </ul>

  </section>

</main>

</body>

</html>
```

OUTPUT:

SEO Best Practices: Optimizing Anchor Tags and Titles

- [Importance of Titles](#)
- [Optimizing Anchor Tags](#)
- [Best Practices](#)

Importance of Titles in SEO

Titles are one of the most important on-page SEO element. Make sure your titles are descriptive, contain relevant keywords, and are unique for each page.

How to Optimize Anchor Tags

Anchor tags, or links, are crucial for SEO as they help search engines understand the content of the linked page. Use descriptive and keyword-rich anchor text for your links. Avoid generic phrases like "click here" or "read more".

Example of a well-optimized anchor tag: [SEO Tips and Tricks](#)

Best Practices for SEO

- Use relevant keywords in your titles and anchor texts.
- Keep your titles under 60 characters to avoid truncation in search results.
- Create unique titles for each page to avoid duplicate content issues.
- Use descriptive and specific anchor texts to provide context.
- Ensure your links are relevant and add value to the content.